

JCS2201 PYTHON PROGRAMMING

UNIT – III COLLECTIONS

UNIT III – COLLECTIONS

Syllabus

- List: Create, Access, Slicing, Negative Indices, List Methods, and comprehensions.
- Tuples: Create, Indexing and Slicing, Operations on tuples.
- Dictionary: Create, add, and replace values, operations on dictionaries.
- Sets: Create and operations on set.



Collections in python

```
a = [1,2,3,4,5] #list
```

```
b = (1,2,3,4,5) #tuple
```

```
c = {1,2,3,4,5} #set
```

```
d = {'a':1,'b':2,'c':3,'d':4,'e':5} #dictionary
```

LISTS

Create List

Access List

List Slicing

Negative Indices in List

List Methods

List comprehensions

PYTHON LISTS

- **List:** A list is a collection of things, enclosed in [] and separated by commas.
- It is a sequence data type which is used to store the collection of data, called as elements / items.
- Python lists store multiple data together in a single variable

Example:

```
fruits = ['apple', 'orange', 'mango']
print(fruits)
num_list = [6, 9, 10, 1, 55]
print(num_list, len(num_list), max(num_list))
address = [18, 'Nivas Flats', 'Pallikaranai', 'Chennai', 600100]
```

OUTPUT ['apple', 'orange', 'mango']
[6, 9, 10, 1, 55] 5 55

CREATING A LIST

- Lists in Python can be created
 - Method1: By just placing the sequence inside the square brackets [].
 - Method2: Creating a list using split() method
 - Method3: Using for / while loop
- **Method1: By just placing the sequence inside the square brackets [].**

Example:

```
list1 = []                                # blank list
fruits = ['apple', 'orange', 'mango']    # list of strings
num_list = [6, 9, 10, 1, 55]             # list of numbers
address = [18, 'Nivas Flats', 'Pallikaranai', 'Chennai', 600100]
                                           # list with mixed type of values
```

CREATING A LIST

- **Method2: Creating a list using split() method**

Example:

```
list1 = input("Enter the list of colors:")
colors = list1.split()
print(colors)
```

OUTPUT

```
Enter the list of colors: red green black white grey
['red', 'green', 'black', 'white', 'grey']
```

CREATING A LIST

• Method3: Creating a list using for / while loop

Example:

```
fruits = []
print("Enter 3 fruit names:")
for i in range(3):
    f = input()
    fruits.append(f)
print(fruits)
fruits.extend('apple', 'mango')
print(fruits)
```

OUTPUT

```
Enter fruit names:
orange
guava
kiwi
['orange', 'guava', 'kiwi']
['orange', 'guava', 'kiwi', 'apple',
'mango']
```

CREATING A LIST

- **Creating a list from string / tuple / set**
- A string / tuple / set can be converted into a list using list() method.
- Converting a tuple to a list makes it mutable, allowing you to change its elements
- When converting a set to a list, the order of elements in the resulting list is not guaranteed

OUTPUT

```
['p', 'y', 't', 'h', 'o', 'n']
[1, 2, 3]
[10, 20, 30]
```

Example:

```
str = 'python'
list1 = list(str)
print(list1)
```

```
tuple1 = (1, 2, 3)
list2 = list(tuple1)
print(list2)
```

```
set1 = {10, 20, 30}
list3 = list(set1)
print(list3)
```

CREATING A LIST

- **Merging lists : Creating a list from 2 or more lists**
- This is done using the concatenation operator +
- Merging lists can also be done using extend() method.

Example:

```
num1 = [1, 2, 3 ]
num2 = [10, 20, 30, 40]
list1 = num1 + num2
print(list1)
num1.extend(num2)
print(num1)
```

OUTPUT

```
[1, 2, 3, 10, 20, 30, 40]
[1, 2, 3, 10, 20, 30, 40]
```

INDEXING A TUPLE

- List elements can be indexed from 0 (from the beginning of the list) or from -1 (from the end of the list).
- The index must be an integer +ve (from beginning) or -ve (from end)
- Using -ve values as index is called as negative indexing.

items	8	13	1	0	19	11	2
Index	0	1	2	3	4	5	6
-ve index	-7	-6	-5	-4	-3	-2	-1

Example:

```
num = [8, 13, 1, 0, 19, 11, 2]
print(num[4])
print(num[-2] ** 2)
num[1] = 29
print(num)
```

OUTPUT

```
19
121
[8, 29, 1, 0, 19, 11, 2]
```

ACCESSING A LIST

- List elements can be accessed using the index operator [].
- The index must be an integer - +ve or -ve
- Nested lists are accessed using nested indexing.
- Example: num = [8, 13, 1, 0, 19, 11, 2]

items	8	13	1	0	19	11	2
Index	0	1	2	3	4	5	6

Example:
num = [8, 13, 1, 0, 19, 11, 2]
print(num[4])
num[1] = 29
print(num)

OUTPUT
19
[8, 29, 1, 0, 19, 11, 2]

ACCESSING A LIST

Example1:
num = [8, 13, 1, 0, 19, 11, 2]
print(num)

OUTPUT1
[8, 29, 1, 0, 19, 11, 2]

Example3:
num = [8, 13, 1, 0, 19, 11, 2]
for value in num:
 print(value)

Example2:
num = [8, 13, 1, 0, 19, 11, 2]
n = len(num)
for i in range(n):
 print(num[i])

OUTPUT2
8
13
1
0
19
11
2

OUTPUT3
8
13
1
0
19
11
2

NEGATIVE INDICES IN LIST

- List elements can be indexed from the end also. But the index values are -ve integers. This is termed as negative indexing.
- Eg. L1 = [8, 13, 1, 0, 19, 11, 2]

L1	8	13	1	0	19	11	2
Index	0	1	2	3	4	5	6
-ve index	-7	-6	-5	-4	-3	-2	-1

Example:

```
num = [8, 13, 1, 0, 19, 11, 2]
print(num[-2] ** 2)
num[-6] = 100
print(num)
```

OUTPUT

```
121
[8, 100, 1, 0, 19, 11, 2]
```

LIST SLICING

- List slicing is a process of getting substrings and sublists.
- It is done similar to string slicing.

SYNTAX:

```
list_name[start_pos : end_pos : step]
```

- To print elements from beginning to a range: *[: Index]*
- To print elements from end: *[:-Index]*
- To print elements from a specific Index till the end: *[Index :]*
- To print the whole list in reverse order: *[: : -1]*

LIST SLICING

- List slicing is a process of getting substrings and sublists.
- It is done similar to string slicing.

SYNTAX:

`list_name[start_pos : end_pos : step]`

- To print elements from beginning to a range: `[ : Index]`
- To print elements from end: `[ :-Index]`
- To print elements from a specific Index till the end: `[Index :]`
- To print the whole list in reverse order: `[ : : -1]`

LIST SLICING

- Example: `L1 = [8, 13, 1, 0, 19, 11, 2]`

items	8	13	1	0	19	11	2
Index	0	1	2	3	4	5	6
-ve index	-7	-6	-5	-4	-3	-2	-1

```
>>>print(L1[ : ])
[8, 13, 1, 0, 19, 11, 2]

>>>print(L1[ : : ])
[8, 13, 1, 0, 19, 11, 2]

>>>i = 5
>>>print(L1[ : i ] + L1[ i : ])
[8, 13, 1, 0, 19, 11, 2]

>>>print(L1[ : 4])
[8, 13, 1, 0]

>>>print(L1[ 3 : ])
[0, 19, 11, 2]

>>>print(L1[2 : 8])
[1, 0, 19, 11, 2]

>>>print(L1[30 : ])
[]
```

LIST SLICING

• Example: L1 = [8, 13, 1, 0, 19, 11, 2]

items	8	13	1	0	19	11	2
Index	0	1	2	3	4	5	6
-ve index	-7	-6	-5	-4	-3	-2	-1

```
>>>print(s[2 : 6 : 2])           >>>print(s[6: 1 : -2])
[1, 19]                          [2, 19, 1]
>>>print(s[ : -1])
[2, 11, 19, 0, 1, 13, 8]
>>>print(L1[6 : 1 : -1])
[2, 11, 19, 0, 1]
```

LIST METHODS

• Python has a set of built-in methods that you can use on lists/arrays.

S.No.	Method	Description
1	append()	Adds an element at the end of the list
2	clear()	Removes all the elements from the list
3	copy()	Returns a copy of the list
4	count()	Returns the number of elements with the specified value
5	extend()	Add the elements of a list (or any iterable), to the end of the current list
6	index()	Returns the index of the first element with the specified value

LIST METHODS

• Python has a set of built-in methods that you can use on lists/arrays.:

S.No.	Method	Description
7	insert()	Adds an element at the specified position
8	pop()	Removes the element at the specified position
9	remove()	Removes the first item with the specified value
10	reverse()	Reverses the order of the list
11	sort()	Sorts the list

LIST METHODS

1. **append()** - Adds an element at the end of the list. Only one element (a value / tuple / list) can be appended at a time

SYNTAX:

`list_name.append(element)`

Example:

```
fruits = [ ]
print("Enter 3 fruit names:")
for i in range(3):
    f = input()
    fruits.append(f)
print(fruits)
fruits.append('apple')
print(fruits)
```

OUTPUT

```
Enter fruit names:
orange
guava
kiwi
['orange', 'guava', 'kiwi']
['orange', 'guava', 'kiwi', 'apple']
```

LIST METHODS

2. clear() - Removes all the elements from the list.
The list is not deleted. Only the list elements are deleted.

SYNTAX:
`list_name.clear( )`

Example:
`fruits = ['orange', 'guava', 'kiwi']`
`print(fruits)`
`fruits.clear()`
`print(fruits)`

OUTPUT
['orange', 'guava', 'kiwi']
[]

LIST METHODS

3. copy() - Returns a copy of the list.
Thus 2 copies of the same list can be obtained. Both will have different id (stored in different locations). The changes made to one list will not affect the other list.

SYNTAX:
`list_name.copy( )`

Example:
`L1 = [6, 2, 9, 5, 1]`
`L2 = L1.copy()`
`print("Original list: ", L1)`
`print("Copied list: ", L2)`
`print(L1 is L2)`
`L2[3] = 100`
`print("Original list: ", L1)`
`print("Copied list: ", L2)`

OUTPUT
Original list: [6, 2, 9, 5, 1]
Copied list: [6, 2, 9, 5, 1]
False
Original list: [6, 2, 9, 5, 1]
Copied list: [6, 2, 9, 100, 1]

LIST METHODS

4. count() - Returns the number of times a specified value is present in a list.

SYNTAX:

`list_name.count(value)`

Example:

```
L1 = [6, 2, 9, 5, 1, 9, 21, 9]
c = L1.count(9)
print(f'9 is present {c} times')
c = L1.count(900)
print(f'900 is present {c} times')
```

OUTPUT

9 is present 3 times
900 is present 0 times

LIST METHODS

5. extend() - Add the elements of a list (or any iterable), to the end of the current list

SYNTAX:

`list_name.extend(values / list / tuple / set)`

Example:

```
fruits = ['orange', 'guava', 'kiwi']
print(fruits)
fruits.extend(['apple', 'mango'])
print(fruits)
num = [10, 50, 90, 30]
fruits.extend(num)
print(fruits)
```

OUTPUT

['orange', 'guava', 'kiwi']
['orange', 'guava', 'kiwi', 'apple', 'mango']
['orange', 'guava', 'kiwi', 'apple', 'mango', 10, 50, 90, 30]

LIST METHODS

6. index() - Returns the index of the first element with the specified value

SYNTAX:

`list_name.index(value)`

Example:

```
L1 = [6, 2, 9, 5, 1, 9, 21, 9]
pos = L1.index(9)
print(f'9 is present at position {pos}')
print(L1.index(100))
```

OUTPUT

9 is present at position 2
ValueError: 100 is not in list

LIST METHODS

7. insert() - Adds an element at the specified position

SYNTAX:

`list_name.insert(position, element)`

Example:

```
L1 = [6, 2, 9, 5, 1, 9, 21, 9]
print(L1)
L1.insert(4, 28)
print(L1)
L1.insert(14, 100)
print(L1)
```

OUTPUT

[6, 2, 9, 5, 1, 9, 21, 9]
[6, 2, 9, 5, 28, 1, 9, 21, 9]
[6, 2, 9, 5, 28, 1, 9, 21, 9, 100]

LIST METHODS

8. pop() - Removes the element at the last position or at any given position. If no position is mentioned as parameter, default position is -1.

SYNTAX:

`list_name.pop(position)`

Example:

```
L1 = [6, 2, 9, 5, 1, 9, 21, 9]
print(L1)
L1.pop() # last element removed
print(L1)
L1.pop(3) # 3rd element removed
print(L1)
```

OUTPUT

```
[6, 2, 9, 5, 1, 9, 21, 9]
[6, 2, 9, 5, 1, 9, 21]
[6, 2, 9, 1, 9, 21]
```

LIST METHODS

9. remove() - Removes the first item with the specified value

SYNTAX:

`list_name.remove(element)`

Example:

```
L1 = [6, 2, 9, 5, 10, 9, 21, 9]
print(L1)
L1.remove(10)
print(L1)
```

OUTPUT

```
[6, 2, 9, 5, 10, 9, 21, 9]
[6, 2, 9, 5, 9, 21, 9]
```

LIST METHODS

10. reverse() - Reverses the order of the list

SYNTAX:

`list_name.reverse()`

Example:

```
L1 = [6, 2, 9, 5, 1, 9, 21, 9]
```

```
print(L1)
```

```
L1.reverse()
```

```
print(L1)
```

OUTPUT

```
[6, 2, 9, 5, 1, 9, 21, 9]
```

```
[9, 21, 9, 1, 5, 9, 2, 6]
```

LIST METHODS

11. sort() - Sorts the list.

If the parameter `reverse = True`, list is sorted in ascending order;

If the parameter `reverse = False`, list is sorted in descending order.

SYNTAX:

`list_name.sort(reverse=True | False, key=function)`

Example:

```
L1 = [6, 2, 9, 5, 1, 9, 21, 9]
```

```
L2 = ['orange', 'guava', 'kiwi', 'apple',  
'mango']
```

```
print(L1)
```

```
L1.sort()
```

```
L2.sort()
```

```
print("Sorted list ", L1)
```

```
print("Sorted fruits list ", L2)
```

```
L1.sort(reverse = True)
```

```
# list sorted in descending order
```

```
print(L1)
```

OUTPUT

```
[6, 2, 9, 5, 1, 9, 21, 9]
```

```
['orange', 'guava', 'kiwi', 'apple',  
'mango']
```

```
Sorted list [1, 2, 5, 6, 9, 9, 9, 21]
```

```
Sorted fruits list ['apple', 'guava',  
'kiwi', 'mango', 'orange']
```

```
[21, 9, 9, 9, 6, 5, 2, 1]
```

Copying and Cloning A List

- **Copying a list:**
- Using `copy()` method
- A shallow copy of a list is created and returned
- Both copies have the same value but have different IDs

Example:

```
L1 = [6, 2, 9, 5, 1]
L2 = L1.copy()
print("Original list: ", L1)
print("Copied list: ", L2)
print(id(L1))
print(id(L2))
L1[2] = 800
print("Original list: ", L1)
print("Copied list: ", L2)
```

OUTPUT

```
Original list: [6, 2, 9, 5, 1]
Copied list: [6, 2, 9, 5, 1]
138285186515776
138285188486784
Original list: [6, 2, 800, 5, 1]
Copied list: [6, 2, 9, 5, 1]
```

Copying and Cloning A List

- **Cloning a list:**
- Using `=` operator
- This operator assigns the old list to the new list.
- Both copies have the same value and same IDs

Example:

```
L1 = [6, 2, 9, 5, 1]
L2 = L1
print("Original list: ", L1)
print("Copied list: ", L2)
print(id(L1))
print(id(L2))
L1[2] = 800
print("Original list: ", L1)
print("Copied list: ", L2)
```

OUTPUT

```
Original list: [6, 2, 9, 5, 1]
Copied list: [6, 2, 9, 5, 1]
138285186515776
138285186515776
Original list: [6, 2, 800, 5, 1]
Copied list: [6, 2, 800, 5, 1]
```

LIST COMPREHENSION

- List:

TUPLES

Create Tuple

Indexing in Tuple

Slicing in Tuple

Operations on tuples

TUPLES

- Python tuples are a type of data structure that is very similar to lists.
- Tuples are used to store multiple items in a single variable.
- Tuple is a collection which is ordered and unchangeable. Allows duplicate members.
- Tuples consist of a collection of 0 or more elements, separated by comma and enclosed within ()
- The main difference between lists and tuples is that tuples are **immutable** (meaning they cannot be changed once they are created).
- This makes them ideal for storing data that should not be modified, such as database records.

TUPLES

Examples:

```
tuple1 = ()                                # blank tuple
fruits = ('apple', 'orange', 'mango', 'apple') # tuple of strings
num_list = (6, 9, 10, 1, 55)               # tuple of numbers
c = len(num_list)
M = max(num_list)
# len( ) function is used to find the number of elements in the tuple
tuple1 = ("apple", "banana", "cherry")
tuple2 = (1, 5, 7, 9, 3)
tuple3 = (True, False, False)
tuple4 = ("abc", 34, True, 40, "male")
```

CREATING A TUPLE

- Tuples in Python can be created
 - Method1: By just placing the sequence inside the brackets().
 - Method2: Creating a tuple using tuple() constructor
 - Method3: By creating a list and then converting it into a tuple using tuple() method

CREATING A TUPLE

- **Method1: By just placing the sequence inside the ().**

Example:

```
tuple1 = ( )           # blank tuple
tup1 = (20)            # tuple with 1 element
num_list = (6, 9, 10, 1, 55)
fruits = ('apple', 'orange', 'mango', 'apple')
print(type(fruits))    # O/P <class 'tuple'>
```

CREATING A TUPLE

- **Method2:** Creating a tuple using tuple() constructor

Example:

```
fruits = tuple(("apple", "banana", "cherry"))  
print(fruits)
```

OUTPUT

```
("apple", "banana", "cherry")
```

CREATING A TUPLE

- **Method3:** By creating a list and then converting it into a tuple using tuple() method.

Example:

```
l1 = [  
l1.append(15)  
l1.append(20)  
print(l1)  
t1 = tuple(l1)  
print(t1)
```

OUTPUT

```
[15, 20]  
(15, 20)
```

INDEXING A TUPLE

- Tuple elements can be indexed from 0 (from the beginning of the tuple) or from -1 (from the end of the tuple).
- The index must be an integer - +ve or -ve.
- Tuple items are indexed, the first item has index [0], the second item has index [1] etc.

items	8	13	1	0	19	11	2
Index	0	1	2	3	4	5	6
-ve index	-7	-6	-5	-4	-3	-2	-1

Example:
num = (8, 13, 1, 0, 19, 11, 2)
print(num[4])
print(num[-2] ** 2)

OUTPUT
19
121

ACCESSING A TUPLE

- Tuple elements can be accessed using the index operator [].
- The index must be an integer - +ve or -ve
- Example: num = (8, 13, 1, 0, 19, 11, 2)

items	8	13	1	0	19	11	2
Index	0	1	2	3	4	5	6
-ve index	-7	-6	-5	-4	-3	-2	-1

Example:
num = (8, 13, 1, 0, 19, 11, 2)
print(num[4])
print(num[-2] ** 2)
num[1] = 29
print(num)

OUTPUT
19
121
TypeError: 'tuple' object does not support item assignment

ACCESSING A TUPLE

- Tuple elements can be accessed using the index operator [] and a loop.

Example:

```
num = (8, 13, 1, 20)
for i in range(len(num)):
    print(num[i])
```

```
for j in num:
    print(j)
```

```
k = 0
while k < len(num):
    print(num[k])
    k = k + 1
```

OUTPUT

```
8
13
1
20
8
13
1
20
8
13
1
20
```

SLICING IN TUPLE

- Tuple slicing is a process of getting substrings and sublists.
- It is done similar to list slicing.

SYNTAX:

```
list_name[start_pos : end_pos : step]
```

- To print elements from beginning to a range: *[: Index]*
- To print elements from end: *[:-Index]*
- To print elements from a specific Index till the end: *[Index :]*
- To print the whole list in reverse order: *[: : -1]*

SLICING IN TUPLE

k = (8, 13, 1, 20, 0, 63, 7, 9, 5)

k	8	13	1	20	0	63	7	9	5
Index	0	1	2	3	4	5	6	7	8
-ve index	-9	-8	-7	-6	-5	-4	-3	-2	-1

EXAMPLE	OUTPUT
print(k[:])	(8, 13, 1, 20, 0, 63, 7, 9, 5)
print(k[: : 3])	(8, 20, 7)
print(k[: -3 :])	(8, 13, 1, 20, 0, 63)
print(k[-1 : -7 : -2])	(5, 7, 0)
print(k[: : -1])	(5, 9, 7, 63, 0, 20, 1, 13, 8)

TUPLE METHODS

- Python has two built-in methods that can be used on tuples.

S.No.	Method	Description
1	count()	Returns the number of times a specified value occurs in a tuple
2	index()	Searches the tuple for a specified value and returns the position of where it was found

TUPLE METHODS

1. count() - Returns the number of times a specified value occurs in a tuple.

SYNTAX:

`tuple_name.count(value)`

Example:

```
t1 = (6, 2, 9, 5, 1, 9, 21, 9)
```

```
c = t1.count(9)
```

```
print(f'9 is present {c} times')
```

```
c = t1.count(900)
```

```
print(f'900 is present {c} times')
```

OUTPUT

9 is present 3 times

900 is present 0 times

TUPLE METHODS

2. index() - Searches the tuple for a specified value and returns the position of where it was found

SYNTAX:

`tuple_name.index(value)`

Example:

```
T1 = (6, 2, 9, 5, 1, 9, 21, 9)
```

```
pos = T1.index(9)
```

```
print(f'9 is present at position {pos}')
```

```
print(T1.index(100))
```

OUTPUT

9 is present at position 2

ValueError: tuple.index(x)
x not in tuple

OPERATIONS ON TUPLES

• The following operations can be performed on tuples.

S. No.	Operation	Description	Example
1.	Tuple Concatenation	To return a tuple created by the concatenation of 2 or more tuples.	>>> (1, 2, 3) + (4, 5, 6) (1, 2, 3, 4, 5, 6)
2.	Iterating through tuple using loop	To traverse through the elements in a tuple, one by one	>>>for k in (1, 5, 2): print(k, end=' ') 1 5 2
3.	Tuple Membership Test	To check whether an element is a present in a tuple or not	>>> 3 in (1, 2, 3) True

OPERATIONS ON TUPLES

1. **Tuple Concatenation** - It is a process of combining 2 or more tuples into a new tuple using + operator.

```
Example:  
T1 = (6, 2, 9, 5)  
T2 = (20, 'Hi', 10, "YES")  
T3 = T1 + T2 + T1  
print(T3)
```

```
OUTPUT  
(6, 2, 9, 5, 20, 'Hi', 10, 'YES', 6, 2, 9, 5)
```

If a tuple has to be concatenated with itself for 1 or more than 1 time, it can be done using * operator as below:

```
Example:  
T4 = T2 * 3  
print(T4)
```

```
OUTPUT  
(20, 'Hi', 10, 'YES', 20, 'Hi', 10, 'YES', 20, 'Hi',  
10, 'YES',)
```

OPERATIONS ON TUPLES

2. Iterating through tuple using loop

- Using a for / while loop the tuple elements can be traversed / accessed one by one.

Example:

```
num = (8, 13, 20)
for i in range(len(num)):
    print(num[i])
```

```
for j in num:
    print(j)
```

```
k = 0
while k < len(num):
    print(num[k])
    k = k + 1
```

OUTPUT

8
13
20
8
13
20
8
13
20

OPERATIONS ON TUPLES

3. Tuple Membership Test

- A process of checking whether an element is present in a tuple or not (a member of a tuple or not)
- It is done using the membership operators – 'in' and 'not in'

Example:

```
greet = ('hi', 'hello', 'welcome', 'good day')
print('hi' in greet)
print('hello' not in greet)
if 'welcome' in T1:
    print('welcome is a greeting word')
```

OUTPUT

True
False
welcome is a greeting word

TUPLE FUNCTIONS

- Python has a set of built-in functions that can be used on tuples.

S.No.	Method	Description
1	all()	Returns true if all elements are true or if tuple is empty.
2	any()	Returns true if any element of the tuple is true. If tuple is empty, return false
3	len()	Returns the number of elements in the tuple (length / size of the tuple)
4	max()	Returns the maximum element of given tuple
5	min()	Returns the minimum element of given tuple

TUPLE FUNCTIONS

S.No.	Method	Description
6	sum()	Sums up the numbers in the tuple
7	sorted()	Input elements in the tuple and return a new sorted list
8	tuple()	Converts an iterable to a tuple.

TUPLE FUNCTIONS

1. **all()** - Returns true if all elements are true or if tuple is empty. 0 and " denote False.

Example:

```
>>>t1 = (6, -2, 9, 5, 1)
>>>t2 = (8, 2, -3, 0, 4)
>>>t3 = ( )
>>>print(all(t1))
True
>>>print(all(t2))
False
>>>print(all(t3))
True
```

SYNTAX:

all(tuple_name)

Example:

```
>>>t1 = ('aa', 'no', 'kv')
>>>print(all(t1))
True
>>>t2 = ('aa', "", 'kv')
>>>print(all(t1))
False
```

TUPLE FUNCTIONS

2. **any()** - Returns true if any element of the tuple is true. If tuple is empty, return false.

Example:

```
>>>t1 = (6, -2, 9, 5, 1)
>>>t2 = (8, 2, -3, 0, 4)
>>>t3 = ( )
>>>print(any(t1))
True
>>>print(any(t2))
True
>>>print(any(t3))
False
```

SYNTAX:

any(tuple_name)

Example:

```
>>>t4 = (0, 0, 0, 0)
>>>print(any(t4))
False
>>>t5 = ('aa', 'no', 'kv')
>>>print(any(t5))
True
```

TUPLE FUNCTIONS

3. len() - Returns the number of elements in the tuple (length / size of the tuple).

SYNTAX:

```
len(tuple_name)
```

Example:

```
>>>t1 = (6, -2, 9, 5, 1)
>>>t2 = ()
>>>print(len(t1))
5
>>>print(len(t2))
0
```

TUPLE FUNCTIONS

4. max() - Returns the maximum element of given tuple.

5. min() - Returns the minimum element of given tuple.

SYNTAX:

```
max(tuple_name)
min(tuple_name)
```

Example:

```
>>>t1 = (6, -2, 9, 5, 1)
>>>t2 = ()
>>>t3 = (19)
>>>print(max(t1))
9
>>>print(max(t2))
ValueError: max() arg is an empty sequence
>>>print(max(t3))
TypeError: 'int' object is not iterable
>>>print(min(t1))
-2
```

TUPLE FUNCTIONS

6. sum() - Returns the sum of all elements in a given tuple.

SYNTAX:

`sum(tuple_name)`

Example:

```
>>>t1 = (6, -2, 9, 5, 1)
>>>t2 = ()
>>>t3 = (19)
>>>t4 = ('race', 'play', 'win', 5, 9)
>>>print(sum(t1))
19
>>>print(sum(t2))
0
>>>print(sum(t3))
TypeError: 'int' object is not iterable
>>>print(sum(t4))
TypeError
```

TUPLE FUNCTIONS

7. sorted() – Takes a tuple as input and returns a sorted list.

SYNTAX:

`sorted(tuple_name)`

Example:

```
>>>t1 = (6, -2, 9, 5, 1)
>>>t2 = ()
>>>t3 = (19)
>>>t4 = ('p', 'y', 't', 'h', 'o', 'n')
>>>print(sorted(t1))
[-2, 1, 5, 6, 9]
>>>print(sorted(t2))
[]
>>>print(sorted(t3))
TypeError: 'int' object is not iterable
>>>print(sorted(t4))
['h', 'n', 'o', 'p', 't', 'y']
```

TUPLE FUNCTIONS

8. **tuple()** – Converts an iterable (a list or a string) to a tuple.

SYNTAX:

```
tuple(list_name)  
tuple(string_name)
```

Example:

```
>>>list1 = [6, -2, 9, 5, 1]  
>>>t1 = tuple(list1)  
>>>print(t1)  
(6, -2, 9, 5, 1)  
  
>>>str1 = 'james'  
>>>t2 = tuple(str1)  
>>>print(t2)  
( 'j', 'a', 'm', 'e', 's')
```

DELETING A TUPLE

Tuples are immutable and hence they do not allow deletion of a part of it.

The entire tuple gets deleted by the use of `del()` method.

SYNTAX:

```
del(tuple_name)
```

Example:

```
>>>t1 = (6, -2, 9, 5, 1)  
>>>del(t1)  
>>>print(t1)  
NameError: 't1' is not defined
```

DELETING A TUPLE

Write a Python program to update a tuple's elements with squares.

```
t1 = (6, -2, 9, 5, 1)
print("Original tuple: ", t1)
list1 = list(t1)
for k in range(len(list1)):
    list1[k] = list1[k] ** 2
t1 = tuple(list1)
print("Updated tuple: ", t1)
```

OUTPUT

Original tuple: (6, -2, 9, 5, 1)

Updated tuple: (36, 4, 81, 25, 1)

Dictionary

Dictionary

Creating Dictionary

Adding elements to a Dictionary

Replacing values in a Dictionary

Operations on dictionaries.

DICTIONARY

- Dictionary is an unordered collection of elements. An element in dictionary has a **key : value pair**.
- All elements in dictionary are placed inside the curly braces { }
- Elements in dictionaries are accessed via keys and not by their position.
- The values of a dictionary can be any data type.
- Keys must be unique and immutable data type
- Eg. Used to organize contact details for friends

Syntax:

```
dict_var = {key1 : value1, key2 : value2, ....}
```

DICTIONARY

Examples:

- `dict1 = {}` # empty dictionary
- `a = {1:"one", 2:"two"}`
- `address = {no: 18, street: "1st Street", area: "KK nagar", city: "Chennai"}`
- `dict = {'Name': 'Priya', 'Marks': [80, 59, 38, 74, 42]}`
- `profile = {"name": "Alice", "age": 25, "occupation": "Teacher"}`

```
>>>print(profile)
{'name': 'Alice', 'age': 25, 'occupation': 'Teacher'}
>>> print(profile['name'])
Alice
>>> profile['age'] = profile['age'] + 10
>>>print(profile['age'])
35
```

CREATING A DICTIONARY

- In Python, a dictionary can be created by using 2 methods:
- **Method 1:** By placing a sequence of elements within curly {} braces, separated by a 'comma'.
- Eg. `profile = {"name": "Alice", "age": 25, "occupation": "Teacher"}`
- **Method 2:** By using the `dict()` constructor with key-value pairs specified within curly braces and as a list of tuples.
- Examples:
 - `profile1 = dict({"name": "Alice", "age": 25, "job": "Teacher"})`
 - `profile2 = dict([("name", "Alice"), ("age", 25), ("job", "Teacher")])`

ADDING ELEMENTS TO A DICTIONARY

- Dictionary elements added one at a time.
- While adding a value, if the key-value already exists, the value gets updated otherwise a new Key with the value is added to the Dictionary.

```
marks = {}
marks["Maths"] = 30
marks["English"] = 45
marks["Phy"] = [24, 28]
marks["Python"] = {"Theory": 45, "Lab": 50}
print(marks)
```

OUTPUT

```
{'Maths': 30, 'English': 45, 'Phy': [24, 28], 'Python': {'Theory': 45, 'Lab': 50}}
```

REPLACING VALUES IN A DICTIONARY

- The values in a dictionary can be updated.

```
marks = {'Maths': 30, 'Phy': [24, 28], 'Python': {'Theory': 45, 'Lab': 50}}
marks["Maths"] = 80
marks["Phy"][0] = 37
marks["Python"]["Theory"] = 49
print(marks)
```

OUTPUT

```
{'Maths': 80, 'Phy': [37, 28], 'Python': {'Theory': 49, 'Lab': 50}}
```

FUNCTIONS IN A DICTIONARY

- The items of the dictionary can be deleted by using the del keyword.

```
fruits = {'apple': 30, 'mango': 45, 'orange': 26}
print(fruits)
del(fruits['mango'])
print(fruits)
```

OUTPUT

```
{'apple': 30, 'mango': 45, 'orange': 26}
{'apple': 30, 'orange': 26}
```

FUNCTIONS IN A DICTIONARY

- The number of elements in a dictionary can be get using len() function.
- The dictionary can be deleted by using the function clear().

```
fruits = {'apple': 30, 'mango': 45, 'orange' : 26}
print(len(fruits))
fruits.clear()
print(fruits)
```

OUTPUT

```
3
{}
```

DICTIONARY METHODS

Method	Description
dict.clear()	Removes all the elements from the dictionary
dict.copy()	Returns a copy of the dictionary
dict.get(key, default = "None")	Returns the value of specified key
dict.items()	Returns a list containing a tuple for each key value pair
dict.keys()	Returns a list containing dictionary's keys
dict.values()	Returns a list of all the values of dictionary
dict.update(dict2)	Updates dictionary with specified key-value pairs
pop()	Removes the element with specified key
popitem()	Removes the last inserted key-value pair

SET

Set

Creating set

Adding elements to a set

Operations on sets

SET

- A set is an **unordered**, **unchangeable** and **unindexed** collection of elements, written in curly brackets {}.
- Set items are unchangeable, but we can remove items and add new items.
- Set items can be of any data type - can store heterogeneous elements.
- Sets are unordered, so we cannot be sure in which order the items will appear.
- Sets do not allow duplicate values.
- The values True and 1 are considered the same value in sets, and are treated as duplicates. Similarly False and 0 are considered same.

Syntax: `set_var = {element1, element2, ....., element-n}`

Eg. `fruits_set = {'apple', 'orange', 'kiwi', 'guava'}`

SET

Eg.

```
set1 = {"apple", "banana", "cherry", "apple", False, True, 0, 56}
print(set1)
print(type(set1))
set2 = set((10, 4, "biscuits", 100, "milk"))
print(set2)
```

OUTPUT:

```
{False, True, 'banana', 'apple', 56, 'cherry'}
<class 'set'>
{100, 4, 10, 'milk', 'biscuits'}
```

ADDING ELEMENTS TO PYTHON SETS

Eg.

```
set1 = {"apple", "banana", "cherry", "apple"}
set1.add("kiwi")
print(set1)
print("Enter 3 more fruit names:")
for x in range(3):
    set1.add(input())
```

Using add method

OUTPUT:

```
{'cherry', 'kiwi', 'apple', 'banana'}
Enter 3 more fruit names:
apple
melon
guava
{'kiwi', 'apple', 'melon', 'cherry', 'guava', 'banana'}
```

OPERATIONS ON PYTHON SETS

Eg.

```
set1 = {"apple", "banana", "cherry"}  
set2 = {"banana", "kiwi", "apple"}  
set3 = set1.union(set2)  
print(set3)  
set4 = set1 | set2  
print(set4)
```

Union operation

OUTPUT:

```
{“apple“, “cherry“, “banana“, “kiwi“}  
{“kiwi“, “cherry“, “apple“, “banana“}
```

OPERATIONS ON PYTHON SETS

Eg.

```
set1 = {"apple", "banana", "cherry"}  
set2 = {"banana", "kiwi", "apple"}  
set3 = set1.intersection(set2)  
print(set3)  
set4 = set1 & set2  
print(set4)
```

Intersection operation

OUTPUT:

```
{“apple“, “banana“}  
{“apple“, “banana“}
```

OPERATIONS ON PYTHON SETS

Eg.

```
set1 = {"apple", "banana", "cherry"}  
set2 = {"banana", "kiwi", "apple"}  
set3 = set1.difference(set2)  
print(set3)  
set4 = set1 - set2  
print(set4)
```

Difference
operation

OUTPUT:

```
{“cherry“}  
{“cherry“}
```

OPERATIONS ON PYTHON SETS

Eg.

```
set1 = {"apple", "banana", "cherry"}  
set1.clear()  
print(set1)
```

Clear operation

OUTPUT:

```
set()
```

OPERATIONS ON PYTHON SETS

Eg.

```
set1 = {"apple", "banana", "cherry"}  
print("cherry" in set1)  
print("apples" not in set1)
```

Membership
operation

OUTPUT:

True

True